

MongoDB na prática

Ambiente via Docker na VM + primeiros comandos

Unidade 7 — NoSQL · Banco de Dados · 2026/2

Prof. M.Sc. Howard Cruz Roatti

Nesta aula

- Subir o **MongoDB** com **Docker** na **VM LabDatabase**
- Conectar com **mongosh** / **Compass** / **DBeaver**
- Modelo: **database** → **collection** → **document**
- **CRUD**: `insertOne`, `find`, `updateMany`, `deleteMany`
- **Índices** e uma pincelada em **aggregation**

 Nada de instalar no Windows: usamos o **container** da VM (isolado, reprodutível, igual em qualquer máquina).

MongoDB via Docker na VM

O MongoDB já vem no `docker-compose.yml` da VM (serviço `mongo`, porta **27017**).

```
# na VM LabDatabase
cd ~/database_services
docker compose up -d mongo      # sobe só o MongoDB
docker compose ps               # confere se está "running"
```

Rodar avulso (sem compose) também funciona:

```
docker run -d --name mongo -p 27017:27017 mongo:7
```

Conectar ao banco

- **mongosh** (shell oficial) — dentro do container:

```
docker exec -it mongo mongosh
```

- **GUIs: MongoDB Compass** ou **DBeaver** →

```
mongodb://localhost:27017
```

💡 O **DBeaver** (mesma IDE do roteiro de SQL) também conecta ao MongoDB — mantém o fluxo de trabalho.

Modelo de dados

```
Servidor
└─ Database (ex.: loja)
    └─ Collection (ex.: produtos) ← equivale à "tabela"
        └─ Document (JSON/BSON) ← equivale à "linha"
```

- **Sem esquema fixo:** documentos da mesma coleção podem ter campos diferentes.
- Todo documento tem um `_id` único (gerado automaticamente se omitido).

```
use loja      // seleciona/cria a database
```

CRUD — Inserir

```
db.produtos.insertOne({
  nome: "Notebook", preco: 3500, tags: ["informática"]
})

db.produtos.insertMany([
  { nome: "Mouse", preco: 80 },
  { nome: "Teclado", preco: 150, sem_fio: true } // campo extra: tudo bem!
])
```

- A coleção `produtos` é **criada automaticamente** na primeira inserção.

CRUD — Consultar (find)

```
db.produtos.find() // todos
db.produtos.find({ preco: { $gt: 100 } }) // preço > 100
db.produtos.find({ tags: "informática" }) // array contém valor

// projeção + ordenação + limite
db.produtos.find(
  { preco: { $lte: 200 } },
  { nome: 1, preco: 1, _id: 0 }
).sort({ preco: -1 }).limit(5)
```

Operador	Significado
<code>\$gt</code> <code>\$gte</code> <code>\$lt</code> <code>\$lte</code>	comparações
<code>\$in</code> <code>\$nin</code>	pertence / não pertence
<code>\$and</code> <code>\$or</code>	lógicos

CRUD — Atualizar e Excluir

```
// atualizar (note: updateOne / updateMany, não "update")
db.produtos.updateOne(
  { nome: "Mouse" },
  { $set: { preco: 90 } }
)
db.produtos.updateMany(
  { preco: { $lt: 100 } },
  { $inc: { preco: 10 } } // incrementa
)

// excluir
db.produtos.deleteOne({ nome: "Teclado" })
db.produtos.deleteMany({ preco: { $gt: 5000 } })
```

⚠ Métodos antigos como `update()` / `remove()` estão **obsoletos** — use `updateOne/Many` e `deleteOne/Many`.

Índices e Aggregation

```
// índice para acelerar buscas por nome
db.produtos.createIndex({ nome: 1 })

// aggregation pipeline: total e média de preço por tag
db.produtos.aggregate([
  { $unwind: "$tags" },
  { $group: { _id: "$tags",
              qtd: { $sum: 1 },
              preco_medio: { $avg: "$preco" } } },
  { $sort: { qtd: -1 } }
])
```

- O **pipeline de agregação** é o "GROUP BY / analytics" do MongoDB.

Boas práticas e próximos passos

- Modele pensando nos **padrões de acesso** (o que a aplicação lê junto, guarde junto).
- Crie **índices** para as consultas frequentes; evite varrer coleções inteiras.
- **Não guarde a senha** no código — use variável de ambiente / `.env`.
- Explore **Aggregation**, **schema validation** e o **MongoDB Atlas** (nuvem, free tier).
- Próximo deck: **De SQL para MongoDB** — traduzindo o que você já sabe.

Dúvidas?

howard.cruz@faesa.br

Próximo →

De SQL para MongoDB